# DLP Assignment 2

## PART 1: Colourization as Regression

Grey input → RGB output (encoder decoder structure)

**Q1. How many convolution layers are present in the regression-based CNN?**

6 convolutional layers: each with a BatchNorm2D and ReLU as activation function

1. downconv1 → encoder layer 1, includes MaxPool2d
2. downconv2 → encoder layer 2, includes MaxPool2d
3. rfconv → bottleneck (no spatial change)
4. upconv1 → decoder layer 1, includes Upsample ×2
5. upconv2 → decoder layer 2, includes Upsample ×2
6. finalconv → output layer(implemented via MyConv2d)

**Q2. Filter sizes and number of filters per layer?**

All convolution layers use a kernel size of 3×3.
No. of Filters in each layer: 32, 64, 64, 32, 3, 3

**Table: (F=no. of filters=32)**

| Layer | In Channels | Out Channels | Kernel | Operations | Spatial Change |
|---|---|---|---|---|---|
| downconv1 | 1 | 32 (F) | 3×3 | Conv + BN + ReLU + MaxPool(2) | 32×32 → 16×16 (MaxPool2) |
| downcov2 | 32 (F) | 64 (2F) | 3×3 | Conv + BN + ReLU + MaxPool(2) | 16×16 → 8×8 (MaxPool2) |
| rfconv | 64 (2F) | 64 (2F) | 3×3 | Conv + BN + ReLU | 8×8 → 8×8 (unchanged) |
| upconv1 | 64 (2F) | 32 (F) | 3×3 | Conv + BN + ReLU + Upsample(2) | 8×8 → 16×16 (Upsample×2) |
| upconv2 | 32 (F) | 3 | 3×3 | Conv + BN + ReLU + Upsample(2) | 16×16 → 32×32 (Upsample×2) |
| finalconv | 3 | 3 | 3×3 | MyConv2d(output) | 32×32 → 32×32 (unchanged) |

**Q3. How many epochs was the CNN model trained for?**

The code provided was trained for 25 epochs. The optimiser is Adam with an initial learning rate of 0.001, batch size=100 and loss function=MSE

**Q4. Training variations**

All variations were trained under identical conditions/hyperparameters. Only the number of training epochs were changed.

| Epochs | Train Loss | Val Loss | Observation |
|---|---|---|---|
| 5 | 0.0136 | 0.0135 | High loss. underfitting clearly visible. Very blurry. |
| 15 | 0.0094 | 0.0094 | Basic colors appear. Good generalization considering low validation loss |
| 25 | 0.0085 | 0.0083 | Best quality jump. sky/grass partially correct. val < train |
| 50 | 0.0071 | 0.0071 | edge sharpening. Val = Train. Not much of difference |
| 100 | 0.0061 | 0.0065 | First overfitting signal (val > train). Also no perceptual gain over 50ep in images |

The optimal stopping point is 25-50 epochs. Val loss is at its lowest relative to train, the visual quality peak has been reached.

**Q5: Discuss the impact of training duration on the model's performance**

The training duration directly influences the model's ability to generalize. With too few epochs for eg 5, the model underfits meaning both training and validation losses remain high, and the output images lack detail and color accuracy. As epochs increase to 15 and 25, losses drop significantly, and the model begins to learn meaningful color mappings. The best balance occurs at between 25-50 epochs, where validation loss reaches its minimum (0.0071), indicating optimal generalization. However, after 100 epochs, validation loss rises to 0.0065 while training loss continues to fall, a clear sign of overfitting meaning the model starts memorizing training data and performs worse on unseen images. Thus, while longer training can improve performance up to a point, excessive training harms generalization.

Improvement Suggestions: (EXTRA)

1. LeakyReLU instead of ReLU
2. MSE + Gradient loss combine
3. Gradient clipping for stable training
4. Kaiming He Initialization for faster convergence

## PART 2: Colourization as Classification

The classification CNN mirrors the RegressionCNN architecture exactly. Same encoder, Same bottleneck, Same decoder structure, same 3×3 kernels and F=32 number of filters

The difference is that the output layer produces 24 channels, rather than 3 RGB regression continuous values. Also the Loss Function used here is CrossEntropyLoss rather than MSE.

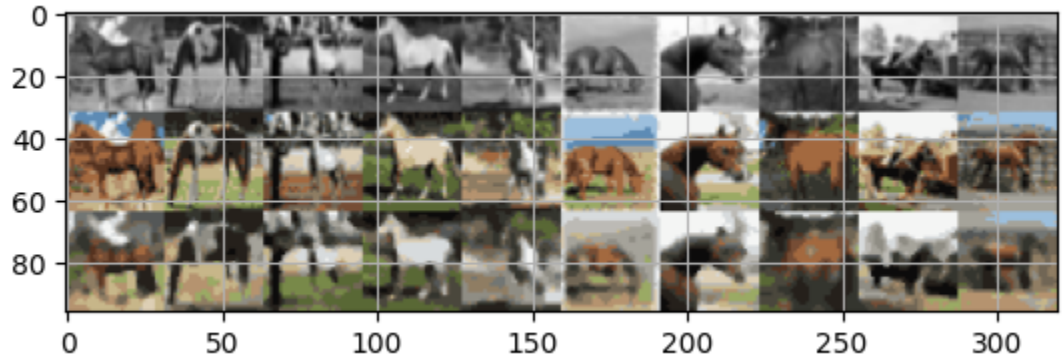
All layers use the custom MyConv2d implementation

| **Layer** | **In** | **Out** | **Kernel** | **Operations** |
|---|---|---|---|---|
| downconv1 | 1 | 32 | 3x3 | MyConv2d→BN→LeakyReLU(0.2)→ MaxPool2 |
| downconv2 | 32 | 64 | 3x3 | MyConv2d→BN→LeakyReLU(0.2)→ MaxPool2 |
| rfconv | 64 | 64 | 3x3 | MyConv2d→BN→LeakyReLU(0.2)→Dropout2d(0.1) |
| upconv1 | 64 | 32 | 3x3 | MyConv2d→BN→LeakyReLU(0.2)→Upsample×2 |
| upconv2 | 32 | 24 | 3x3 | MyConv2d→BN→LeakyReLU(0.2)→Upsample×2 |
| finalconv | 24 | 24 | 3x3 | MyConv2d only |

**Q: Comparison: Regression vs Classification outputs**

**Regression**

**Classification**




Classification is giving 42% Accuracy

The two losses are measuring completely different things. So their loss values are not directly comparable in magnitude.

| | **Regression** | **Classification** |
|---|---|---|
| Loss Type | MSE | CrossEntropy |
| Output Space | Continuous RGB [0,1] | Discrete: 24 palette colours |
| Observation | Soft, Blurry, grey background, poor quality | Good Colors, Sharper, Background visible, Better |

Classification model performed much better than the Regression model

**Qualitative analysis:**
The classification model produces noticeably more vivid and plausible colours than the regression model. This is because instead of averaging over all training colours (which causes greyness), it selects the single most likely colour cluster for each pixel. The 24 k-means clusters

represent the dominant colours of horse images, so the classification decision boundary directly encodes what colour that region of a horse image is most likely to be.

However, the classification model introduces a different artefact: colour blocking. Since the palette is restricted to 24 colours, smooth colour gradients in nature (sky, grass, the horse coat) are restricted into discrete bands. This is analogous to colour quantisation in early computer graphics. The regression model, despite being desaturated, produces smoother spatial gradients.

In terms of validation accuracy, the classification model achieves 42% accuracy. This number is less informative than it appears: adjacent k-means clusters are often very similar in RGB space, so a prediction one cluster away from the ground truth is perceptually nearly correct but counted as wrong by this metric

The single most telling sample is the white/grey horse. Regression predicts it brown, it has no mechanism to say "I'm uncertain, this could be light." Classification at least lands on the light cluster because the discrete palette forces a committed decision. That one sample captures the entire story of why treating colorization as a regression problem is fundamentally limited regardless of architecture or training duration.

# Part 3: Skip Connections (UNet)

**UNet Architecture :**
The UNet extends the classification CNN by introducing symmetric skip connections that concatenate encoder feature maps directly into the corresponding decoder stages.

First 3 layers remain exactly the same, but the remaining last 3 layers(2 decoder and 1 output) get extra previous layer inputs.

Upconv1: rfconv + downconv2
Upconv2: upconv1 + downconv1
Final: upconv2 + input

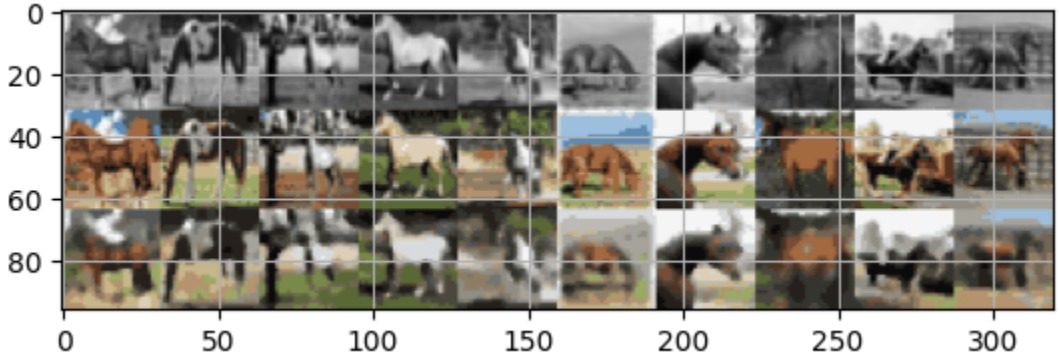
**Q1. Did skip connections improve validation loss and accuracy?**

Yes. The UNet consistently achieves lower validation loss and higher pixel accuracy than the plain CNN across all epochs. The training loss also converges much much faster.

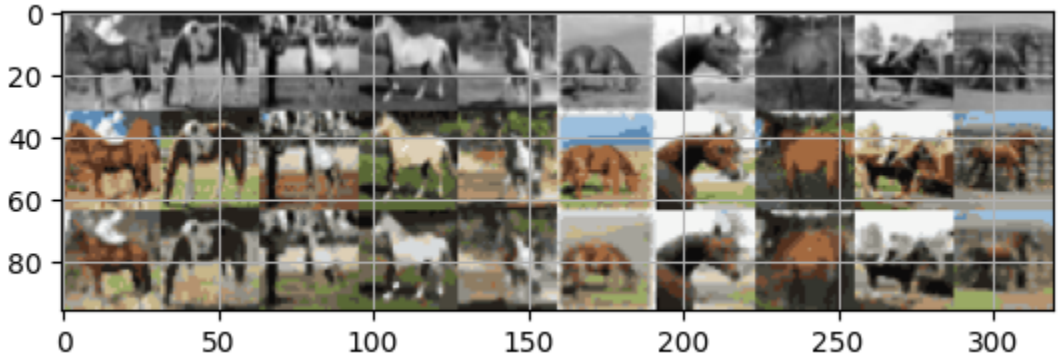
| (at 25 epochs) | Simple CNN | UNet |
|---|---|---|
| **Train Loss** | `1.5963` | `1.3050` |
| **Val Loss** | `1.5636` | `1.3025` |
| **Accuracy** | `41.9%` | `50.1%` |

**Q2: Did skip connections lead to qualitatively better outputs?**

**Simple CNN**




**UNet**



Yes, UNet produces qualitatively better outputs, but the margin is modest at 32×32. The improvements are real and consistent (cleaner boundaries, more coherent regions, better fg/bg separation) but not dramatic. The quantitative gap in val loss and accuracy is a more reliable signal of improvement than visual inspection alone at this resolution.

**Q3: Two reasons why skip connections enhance CNN performance?**

1. **Preservation of spatial detail through the bottleneck**. Without skip connections, max-pooling throws away exact positions of edges and textures. The decoder then has to guess those details from a compressed bottleneck, which results in blurry results. Skip connections hand-deliver the original feature maps from each encoder level straight to the matching decoder level, so no detail is lost.
2. **Better gradient flow during backpropagation.** In a deep network, gradients have to travel through many layers before they reach the early encoder. That often causes them to vanish, making the first layers learn slowly. Skip connections act like shortcuts: gradients can bypass the decoder and flow directly to the encoder, keeping them strong and speeding up convergence.

# Part 4: Visualising Intermediate Activations

Visualizing CNN Activations for Test Examples:

- Describe how the activations in the early layers differ from those in the deeper layers of the CNN. You do not need to include the output images in your write-up—only a detailed description of your observations.

conv1: fine-grained local detectors
All 32 maps at 16×16 resolution are densely active. Each channel highlights a specific low-level structure, curved back, vertical legs, flat background. The horse shape is clearly recognisable. This layer preserves the most spatial detail.

conv2: more abstract
Resolution halves to 8×8. Roughly half the maps are mostly dark. the active ones show patterns (entire body region instead of individual edges). The horse silhouette remains, but now looks like a rough mask.

conv3: highly abstract, sparse
Still at 8×8, but no horse shape remains. This is the information bottleneck: the network has compressed the image into a tiny set of abstract cues that indicate “foreground vs background,” “textured vs smooth,” etc. All colour decisions must flow through this narrow channel.

conv4: spatial structure re emerges
After upsampling back to 16×16, activity returns. Maps now represent broad semantic regions, top half (sky), bottom half (ground), central blob (horse). Boundaries are still diffuse, but the layout is being reconstructed from the bottleneck.

conv5: 24 colour probability maps at full resolution
At 32×32, each of the 24 channels acts as a probability map for one colour cluster. Taking the argmax across these maps gives the final colour prediction.

**In summary, Early layers (conv1–2) produce dense, spatially detailed activations that preserve the horse’s silhouette and respond to low-level features like edges and textures. Deeper layers (conv3–5) become progressively more abstract: Conv3 is a sparse bottleneck with only a few active cells encoding high-level semantics. Conv4 reconstructs semantic regions (sky, ground, body) with diffuse boundaries. Conv5 outputs dense 24-channel probability maps that directly determine the final colour per pixel.**

Visualizing Colorization UNet Activations for Test Examples:

- Discuss the differences between the activations in the UNet and those observed in the CNN. Provide a detailed analysis based on your observations, without including the output images.

In the plain CNN, the early encoder layers (conv1 and conv2) still capture edges and shapes, but the bottleneck (conv3) becomes extremely sparse. Most maps show only a few bright spots, losing all sense of the horse's outline. The decoder then tries to reconstruct colour from that sparse information, resulting in blurry, bleeding boundaries.

In the UNet, the early encoder activations look similar to the CNN, but with one key difference: because the decoder will later receive a direct copy of these encoder features via skip connections, the network doesn't have to cram every spatial detail through the bottleneck. The bottleneck maps (conv3) become cleaner. They focus on global semantics instead of "where exactly". When we look at the decoder layers in the UNet, the improvement is dramatic. At conv4, the region boundaries are much sharper than in the CNN. At conv5 (the final 24 colour probability maps), the UNet's maps are almost binary: one colour lights up clearly inside the horse, another lights up clearly outside, with almost no overlap. This produces crisp, well-defined edges in the final colourisation.

So the main difference is that skip connections let the network separate "what" from "where". The bottleneck handles global colour decisions, while the encoder features provide precise edge information directly to the decoder. The result is much sharper activations and cleaner colour boundaries.

(images uploaded inside github folders)

## Part 5: Theoretical Questions

1. The loss functions and evaluation metrics provided in the supplementary code are defined at the pixel level. However, pixel-level measures often do not align well with human perception of visual quality. How can the evaluation process be improved to better reflect human judgment? Reference [2] may provide insights into addressing this issue.

Pixel-level losses(like MSE or cross entropy) compare each pixel individually, but they don't care whether the overall image looks realistic. For example, a blurry image can have low pixel error, yet it looks terrible to us.
A better approach, inspired by the reference paper, is to use perceptual losses. Instead of comparing raw pixels, you feed both the predicted and the target image through a pre-trained network like VGG and compare their high level feature maps. These features capture structure, texture, and semantic content that humans actually notice. Minimizing perceptual loss yields images that are sharper and more visually pleasing. You can also use metrics like SSIM(structural similarity) or LPIPS(learned perceptual similarity), or even run a user study to directly measure human preference.

2. Our convolutional neural networks were trained on 32×32 input and output images. However, during testing, the desired output size is often different. Explain how the trained models can be adapted to colorize test images larger than 32×32.

Our model was trained on 32×32 patches, but it is fully convolutional, meaning it has no fully connected layers. That means it can be applied to images of any size, as long as the spatial dimensions are handled correctly.
One simple way is to slide a 32×32 window over the large image, colorize each patch, and stitch the results together (averaging overlaps to avoid seams).
Alternatively, because all layers (convolutions, pooling, upsampling) work on any spatial size, you can simply feed the entire large image through the network, and the architecture will produce an output of the same size. However, you must ensure that the down-and-upsampling (max-pooling and upsampling) are compatible: if the input size is not a multiple of the total downsampling factor (4 in our case), you may need to pad or resize. In practice, you can pad the image to a multiple of 4, run the model, then crop back.